Integración de datos

Tarea Práctica - Clase 6

Sergio Enrique Morel Peralta
Facultad Politécnica - UNA, 03.04.2026

1. Tarea 6: Testing y Documentación con dbt

Esta tarea extiende el proyecto dbt de la Tarea 5 agregando una capa completa de **testing** y **documentación**. Se trabaja sobre el mismo repositorio y el mismo proyecto `mi_proyecto_dbt`, que transforma datos de OpenWeather y GitHub disponibles en MotherDuck (`md:airbyte_curso`). El objetivo es garantizar la calidad de los datos mediante tests genéricos, tests de `dbt-expectations` y tests singulares personalizados, y documentar todos los modelos y columnas clave. Todos los tests deben pasar con `dbt build`.

1.1. Tests genéricos

Los tests genéricos se declaran en archivos `_models.yml` dentro de cada capa. Se definieron 11 tests en total, distribuidos en staging, intermediate y marts.

1.1.1. Staging

Los modelos de staging tienen tests de unicidad y nulidad sobre sus claves primarias naturales, además de `not_null` en columnas críticas.

```yaml
models:
  - name: stg_weather__forecast
    columns:
```

```yaml
      - name: dt_unix          # PK natural
        tests: [unique, not_null]
      - name: fecha
        tests: [not_null]

  - name: stg_github__stargazers
    columns:
      - name: usuario_github_id   # PK natural
        tests: [unique, not_null]
      - name: usuario_login
        tests: [not_null]
      - name: repositorio_nombre_completo
        tests: [not_null]

  - name: stg_github__branches
    columns:
      - name: rama_nombre          # PK natural
        tests: [not_null]
      - name: repositorio_nombre_completo
        tests: [not_null]
      - name: rama_commit_sha
        tests: [not_null]
```

1.1.2. Intermediate

El modelo `int_github_actividad` incluye un test de `relationships` que verifica que todo `repositorio_nombre_completo` presente en los stargazers exista en la tabla de branches.

```yaml
models:
  - name: int_github_actividad
    columns:
      - name: repositorio_nombre_completo
        tests:
          - not_null
          - relationships:
              to: ref('stg_github__branches')
              field: repositorio_nombre_completo
```

1.1.3. Marts

Los modelos mart tienen tests de unicidad y nulidad sobre sus claves primarias surrogate y un test de `accepted_values` sobre `parte_dia`.

```yaml
models:
  - name: obt_pronostico
    columns:
      - name: pronostico_id
        tests: [unique, not_null]
      - name: pais
```

```yaml
        tests:
          - accepted_values:
              values: ['PY']
      - name: fecha
        tests: [not_null]
      - name: temperatura_c
        tests: [not_null]
      - name: humedad_pct
        tests: [not_null]

  - name: obt_github_actividad
    columns:
      - name: estrella_id
        tests: [unique, not_null]
      - name: usuario_login
        tests: [not_null]
      - name: repositorio_nombre_completo
        tests: [not_null]
      - name: starred_at
        tests: [not_null]
```

1.1.4. Resultado

Los tests genéricos pasaron exitosamente. El resultado consolidado con todos los tests del proyecto ejecutando `dbt run && dbt test`:

```
dbt run:  PASS=6  WARN=0  ERROR=0  TOTAL=6
dbt test: PASS=34 WARN=0  ERROR=0  TOTAL=34
```

Incluye el nuevo test `accepted_values_stg_weather__forecast_parte_dia__d__n` (test 2 de 34), que ahora pasa correctamente gracias a la corrección en el staging.

1.2. Tests de dbt-expectations

La librería `dbt-expectations` (ya declarada en `packages.yml` desde la Tarea 5) provee tests estadísticos y de calidad de datos más expresivos que los genéricos. Se definieron 3 tests distribuidos en staging y marts.

1.2.1. Recuento de filas en staging

Se verificó que ambas tablas de staging de GitHub tengan al menos una fila, y que el pronóstico weather no supere las 40 filas esperadas de la API (5 días × 8 intervalos de 3 h).

```yaml
# staging/_models.yml

- name: stg_weather__forecast
  tests:
    - dbt_expectations.expect_table_row_count_to_be_between:
```

```yaml
        min_value: 1
        max_value: 40

- name: stg_github__stargazers
  tests:
    - dbt_expectations.expect_table_row_count_to_be_between:
        min_value: 1
        max_value: 100
```

1.2.2. Rango de valores en marts

Se verificó que `prob_precipitacion` en `obt_pronostico` siempre esté en el rango válido de la API de OpenWeather (0.0 a 1.0). Esta columna proviene de `pop` (campo directo, no anidado en struct), por lo que su tipo es numérico nativo en MotherDuck.

```yaml
# marts/_models.yml

- name: obt_pronostico
  columns:
    - name: prob_precipitacion
      tests:
        - dbt_expectations.expect_column_values_to_be_between:
            min_value: 0
            max_value: 1
```

1.3. Singular tests

Los singular tests son archivos `.sql` en `tests/` que implementan reglas de negocio personalizadas. dbt los ejecuta como consultas: el test pasa si la consulta retorna 0 filas (no hay violaciones).

1.3.1. `assert_parte_dia_valida`

Valida que `parte_dia` en `obt_pronostico` solo contenga los valores `'d'` (día) o `'n'` (noche) definidos por la API de OpenWeather.

La implementación de los tests reveló un problema en el modelo de staging: Airbyte almacena el campo `sys` de OpenWeather como columna de tipo `JSON` en MotherDuck (visible como `{"pod":"n"}`). Al acceder `sys.pod`, DuckDB retorna un valor de tipo `JSON`, lo que causaba que `accepted_values` fallara con un error de conversión al intentar comparar con strings planos.

La corrección correcta es normalizar el tipo en la capa de staging — que es justamente donde deben resolverse los problemas de tipo antes de que los datos lleguen a capas downstream. Se actualizó `stg_weather__forecast.sql`:

```sql
-- antes
sys.pod                          AS parte_dia,

-- después (corregido en Tarea 6 a partir de los tests)
json_extract_string(sys.pod, '$') AS parte_dia,
```

Con `parte_dia` ya normalizado a `VARCHAR` desde staging, el singular test se simplifica a una comparación directa, y se agregó también el `accepted_values` genérico en `staging/_models.yml`:

```sql
-- tests/assert_parte_dia_valida.sql
SELECT *
FROM {{ ref('obt_pronostico') }}
WHERE parte_dia NOT IN ('d', 'n')
```

1.3.2. assert_temperatura_rango_valido

Valida que la temperatura mínima nunca supere a la máxima en ningún intervalo del pronóstico. Es una regla de negocio meteorológica fundamental.

```sql
SELECT *
FROM {{ ref('obt_pronostico') }}
WHERE temp_min_c > temp_max_c
```

1.3.3. assert_repositorio_formato_valido

Valida que `repositorio_nombre_completo` en `obt_github_actividad` tenga el formato correcto `owner/repo`: ambas partes derivadas por `SPLIT_PART` deben ser strings no vacíos. Un formato inválido (ej. `/repo`, `owner/`, `/`) produciría partes vacías e indicaría datos corruptos del conector de GitHub en Airbyte.

```sql
SELECT *
FROM {{ ref('obt_github_actividad') }}
WHERE repositorio_propietario = ''
   OR repositorio_nombre = ''
```

1.3.4. Resultado

```
dbt run:  PASS=6  WARN=0  ERROR=0  TOTAL=6
dbt test: PASS=34 WARN=0  ERROR=0  TOTAL=34
```

1.4. Documentación de modelos y columnas

La documentación se declara en los archivos `_models.yml` de cada capa mediante los campos `description` a nivel de modelo y de columna. dbt la incorpora al catálogo generado por `dbt docs generate`, haciéndola navegable desde el DAG.

1.4.1. Staging

Cada modelo documenta su propósito, la fuente cruda que consume y las particularidades de tipo heredadas de Airbyte.

Modelo	Descripción
`stg_weather__forecast`	Limpieza de `weather.weather`. Extrae campos de fecha/hora, desanida structs y normaliza tipos JSON. Columnas clave: `dt_unix` (PK), `fecha`, `parte_dia`, `temperatura_c`, `humedad_pct`, `prob_precipitacion`.
`stg_github__stargazers`	Limpieza de `github.stargazers`. Expone datos del usuario y fecha del evento de estrella. Columnas clave: `usuario_github_id` (PK), `usuario_login`, `repositorio_nombre_completo`, `starred_at`.
`stg_github__branches`	Limpieza de `github.branches`. Expone nombre, SHA del commit y estado de protección de cada rama. Columnas clave: `rama_nombre` (PK), `repositorio_nombre_completo`, `rama_commit_sha`.

1.4.2. Intermediate

Modelo	Descripción
`int_github_actividad`	Une `stg_github__stargazers` con `stg_github__branches` mediante LEFT JOIN sobre `repositorio_nombre_completo`. Deriva `repositorio_propietario` y `repositorio_nombre` partiendo el nombre completo por `/`. Columnas clave: `usuario_github_id`, `repositorio_nombre_completo`, `repositorio_propietario`, `repositorio_nombre`.

1.4.3. Marts

Modelo	Descripción
`obt_pronostico`	OBT del pronóstico meteorológico. Aplana todos los atributos de `stg_weather__forecast` y agrega `pronostico_id` como PK surrogate. Materializada como tabla en MotherDuck. Columnas clave: `pronostico_id` (PK), `fecha`, `parte_dia`, `ciudad`, `pais`, `temperatura_c`, `humedad_pct`, `prob_precipitacion`.
`obt_github_actividad`	OBT de actividad GitHub. Combina evento de estrella, datos del usuario y rama principal del repositorio. Agrega `estrella_id` co-

Modelo	Descripción
	mo PK surrogate. Materializada como tabla en MotherDuck. Columnas clave: `estrella_id` (PK), `usuario_login`, `repositorio_nombre_completo`, `rama_principal_nombre`, `starred_at`.

1.5. DAG con documentación generada

El DAG actualizado refleja los singular tests como nodos del grafo de linaje, visibles al filtrar por cada test en `dbt docs serve`. Los tests aparecen en color magenta (distinto a los modelos en azul y las fuentes en verde), lo que permite identificar rápidamente qué nodos son tests y sobre qué modelos actúan.

`assert_parte_dia_valida`: valida `obt_pronostico`, cuyo linaje completo es `weather.weather -> stg_weather__forecast -> obt_pronostico -> assert_parte_dia_valida`.

Figura 1: DAG — `assert_parte_dia_valida` (`+assert_parte_dia_valida`)

`assert_temperatura_rango_valido`: valida `obt_pronostico` por el mismo linaje.

Figura 2: DAG — `assert_temperatura_rango_valido` (`+assert_temperatura_rango_valido`)

`assert_repositorio_formato_valido`: valida `obt_github_actividad`, cuyo linaje completo es `github.branches` + `github.stargazers` -> staging -> `int_github_actividad` -> `obt_github_actividad` -> `assert_repositorio_formato_valido`.

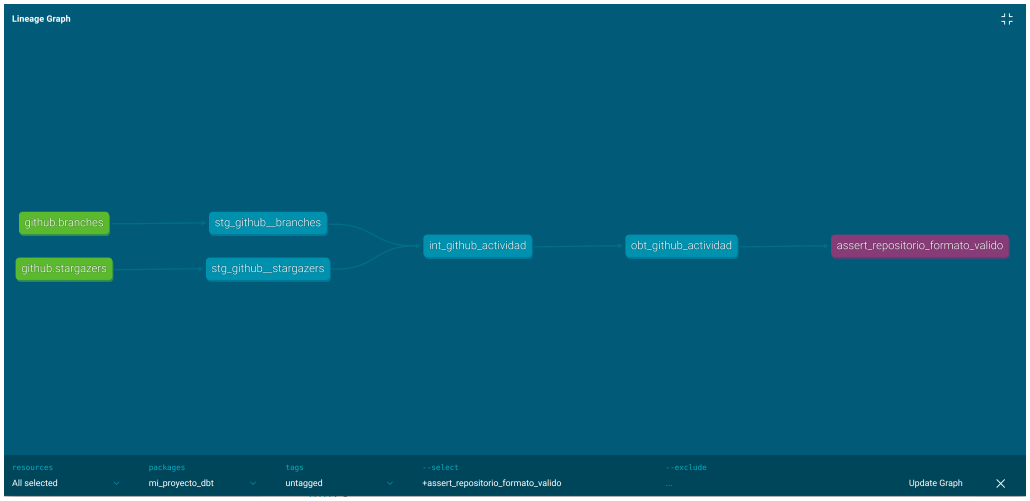

Figura 3: DAG – `assert_repositorio_formato_valido` (`+assert_repositorio_formato_valido`)

1.6. Resultado final: `dbt build`

`dbt build` ejecuta modelos y tests en orden topológico en un solo comando. Los tests de cada modelo se ejecutan inmediatamente después de que ese modelo es materializado, garantizando que los datos estén validados antes de ser consumidos por capas downstream.

```
Finished running 2 table models, 34 data tests, 4 view models in 32.29s.
PASS=40  WARN=0  ERROR=0  SKIP=0  TOTAL=40
```

```
23:43:20  Finished running 2 table models, 34 data tests, 4 view models in 0 hours 0 minutes and 32.29 seconds (32.29s).
23:43:20
23:43:20  Completed successfully
23:43:20
23:43:20  Done. PASS=40 WARN=0 ERROR=0 SKIP=0 NO-OP=0 TOTAL=40
```

Figura 4: Resultado de `dbt build` – PASS=40